# HEXACORE – Vistas vigentes de la arquitectura

Revisión 2/11/2026. ADR-16; fuentes de código y evidencia en ESTADO-MIGRACION. Se migra funcionalidad existente; no se completan CU pendientes. Las versiones anteriores se conservan en Historico/ y no describen la implementación vigente. El corte habitual y la integración a develop siguen pendientes. Los requisitos del SRS no cambian.

## Índice

**1 Contexto** **2**

**2 Panorama de sistemas vigente** **2**

**3 Vista de contenedores** **2**

**4 Vista de componentes** **3**

**5 Vista de procesos** **5**

**6 Vista física** **6**

**7 Modelo de datos vigente** **7**

**8 Reparto, evidencia y pendientes** **8**

## 1. Contexto

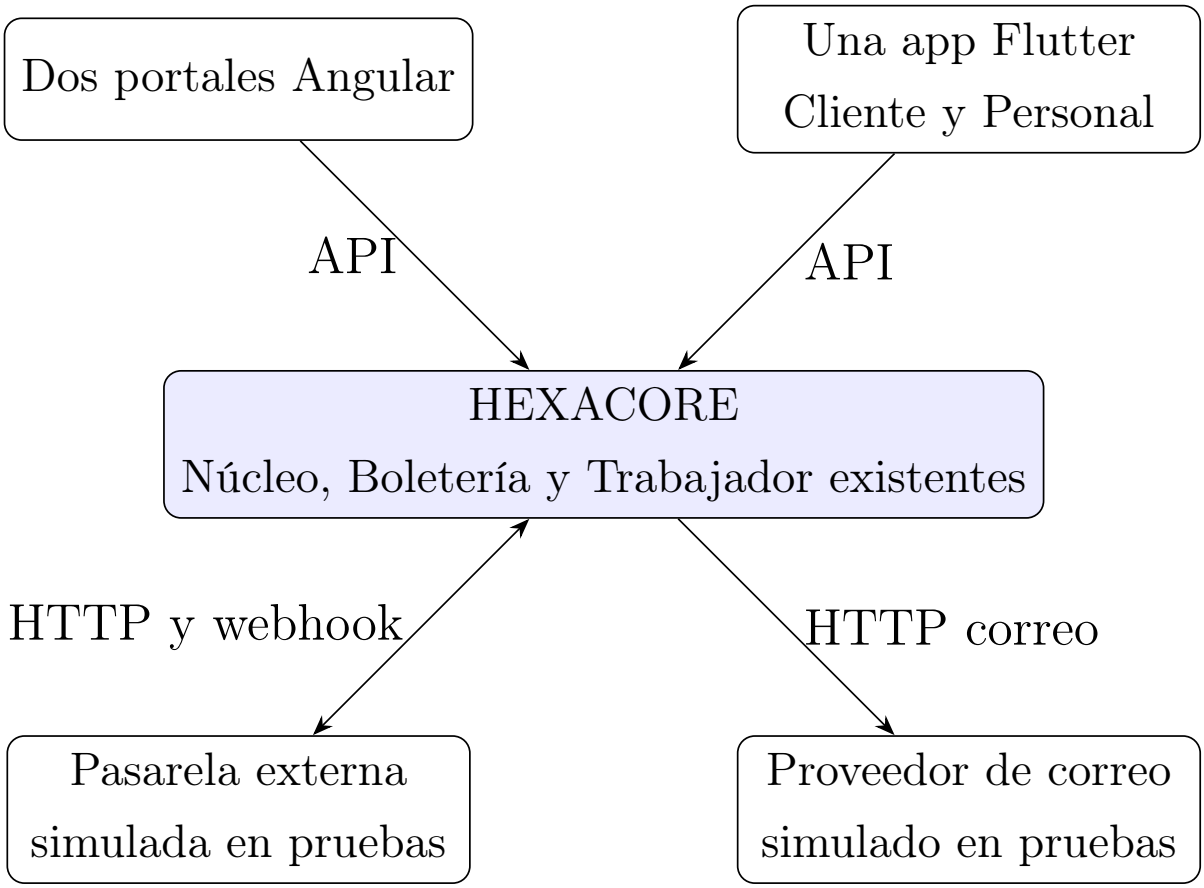



Figura 1: Interfaces y sistemas externos; proveedores simulados en pruebas.

## 2. Panorama de sistemas vigente

El panorama confirmado contiene HEXACORE y los proveedores externos de pago/correo, representados en el contexto anterior. CRM, contabilidad externa y analítica del panorama histórico siguen siendo propuestas sin integración implementada.

## 3. Vista de contenedores

**Vigencia: ADR-16, rama de arquitectura.** Tres programas ejecutables: Núcleo, Boletería y Trabajador. Operación es el cuarto programa objetivo y sigue sin implementar. Los módulos ausentes no se presentan como servidores terminados. La configuración y las imágenes se validan en infraestructura aislada; el entorno habitual sigue pendiente de corte. Estado y ejecuciones reales en `ESTADO-MIGRACION.md`, apartados 23–24.

Dos portales Angular y una app Flutter unificada para Cliente/Personal consumen el

gateway. Sus pantallas o datos simulados no demuestran backend. Gateway verifica JWT con pública y consume el turno; los módulos autorizan y comprueban revocación. Identidad conserva firma/renovación. No existen servidores separados de Pagos, Autenticación, Alimentos, Personal ni los cinco componentes de Boletería.

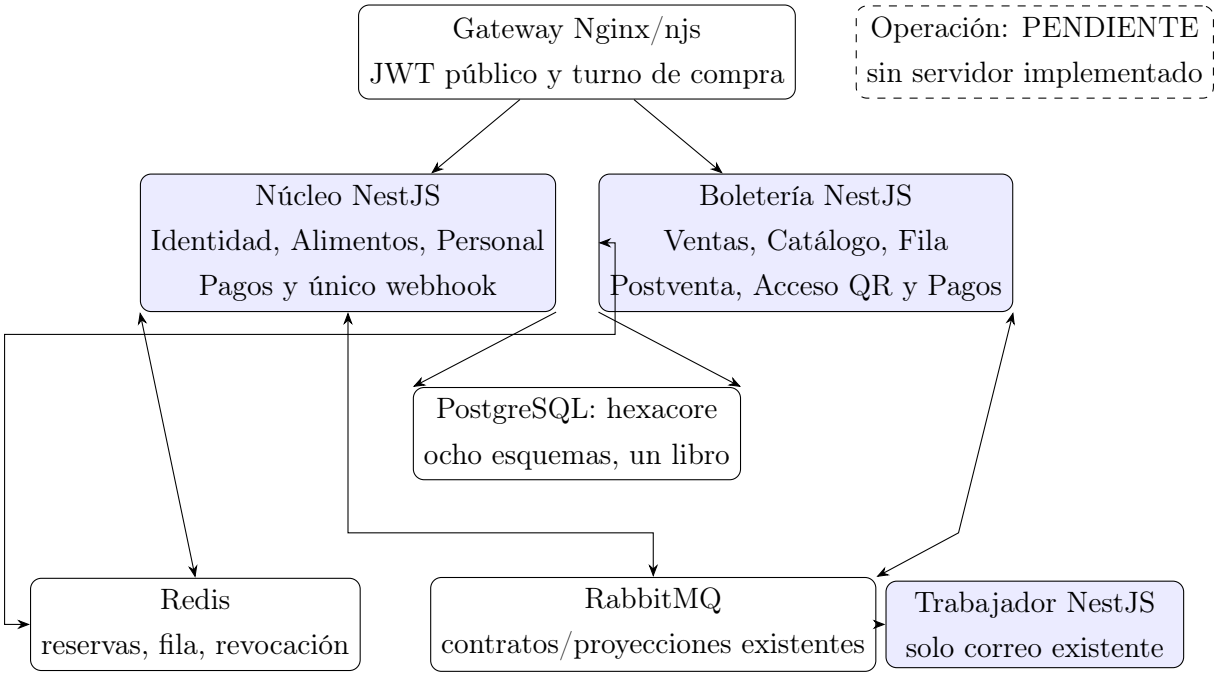



Figura 2: Contenedores vigentes. Azul: programas existentes; discontinuo: objetivo pendiente.

## 4. Vista de componentes

Cada programa tiene bootstrap real en `App/backend/apps`. Las reglas tienen una sola fuente en `modulos`; dependencias y lockfile comunes, Node 22.22.3, Nest 11/TypeScript 5.9. Los compiladores Angular están fuera del workspace.

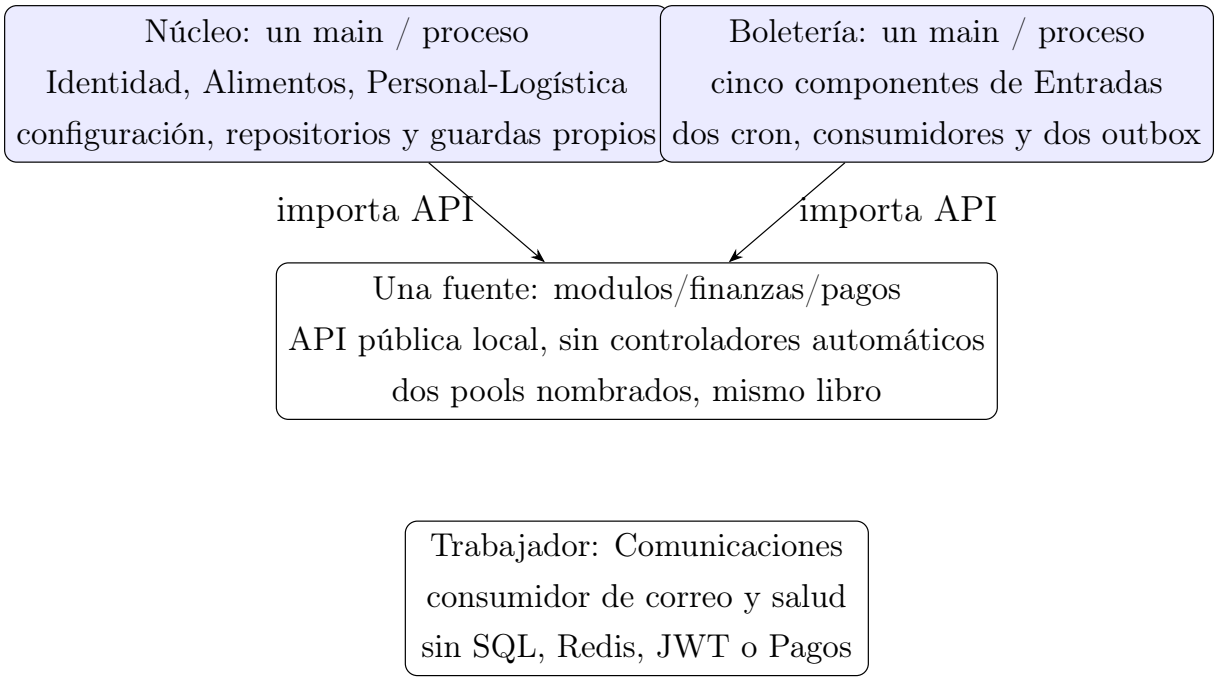
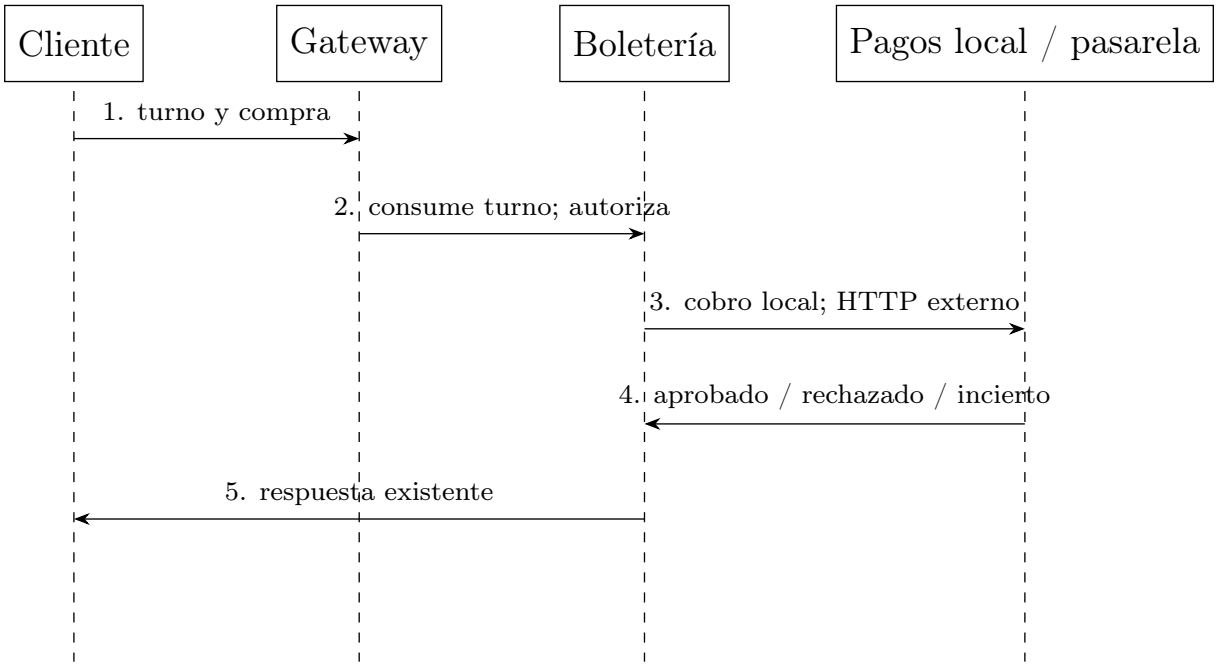



Figura 3: Composición actual. Importar Pagos no publica cobros o reembolsos HTTP.

Núcleo reúne cuentas, sesiones, administración, pedidos y logística; Boletería reúne cartelera, fila, compras, reventa e ingresos. Se conservan rutas `/api/v1`, DTO, estados, errores, cookies, RS256 y reglas previas. El único webhook financiero es `POST /api/v1/pagos/webhook` en Núcleo, con rawBody/HMAC de bytes originales. Boletería no necesita privada JWT ni secreto de webhook. Trabajador consume correo, sin SQL/Redis/JWT/Finanzas y sin endpoints de negocio.

# 5. Vista de procesos




Libro compartido; QR/ingreso convergen por eventos.
Webhook posterior en Núcleo no finaliza negocio pendiente (F12).

Figura 4: Compra existente; no representa una Saga nueva ni finalización F12.

Alimentos y Entradas llaman a la API pública local de Pagos; solo la pasarela mantiene HTTP externo. Postventa usa la API local de Ventas, conservando guardas y efectos. Compartir proceso/base no reúne automáticamente sus transacciones. Ventas conserva autoridad de propietario/QR/historial; Acceso QR mantiene proyección y registro de ingreso. Continúan eventos, compensaciones y dos outbox. La invalidación inmediata del QR no se acredita con una espera de convergencia.

| Contrato existente | Dueño / proceso | Alcance comprobado |
|---|---|---|
| Correo pendiente | Identidad / Núcleo; Comunicaciones / Trabajador | Cinco correos por RabbitMQ; intentos/deduplicación en memoria. |
| Compra, entrada e ingreso | Ventas y Acceso QR / Boletería | Proyecciones y outbox conservados; no consistencia inmediata nueva. |
| Cambio/asignación de turno | Personal-Logística / Núcleo | Notificación persistida; listado global F14. |

| Contrato existente | Dueño / proceso | Alcance comprobado |
|---|---|---|
| Resultado financiero | Webhook / Núcleo | Libro y publicación; finalizar pendientes sigue F12. |
| Pedido confirmado | Alimentos / Núcleo | Publicación existente; recepción final pendiente. |
| Evacuación y alertas | Operación, pendiente | No hay consumidor o caso terminado que migrar. |

Tabla 1: Mensajería vigente; no todo publicador tiene outbox.

Cron de reservas/publicaciones, sondeo de alimentos y consumidores de logística/ Entradas permanecen con sus reglas/repositorios en Núcleo/Boletería. Solo el correo se traslada al Trabajador. SIGTERM cierra pools, cron, consumidores y conexiones; el correo en vuelo conserva su posible reentrega (F15).

## 6. Vista física

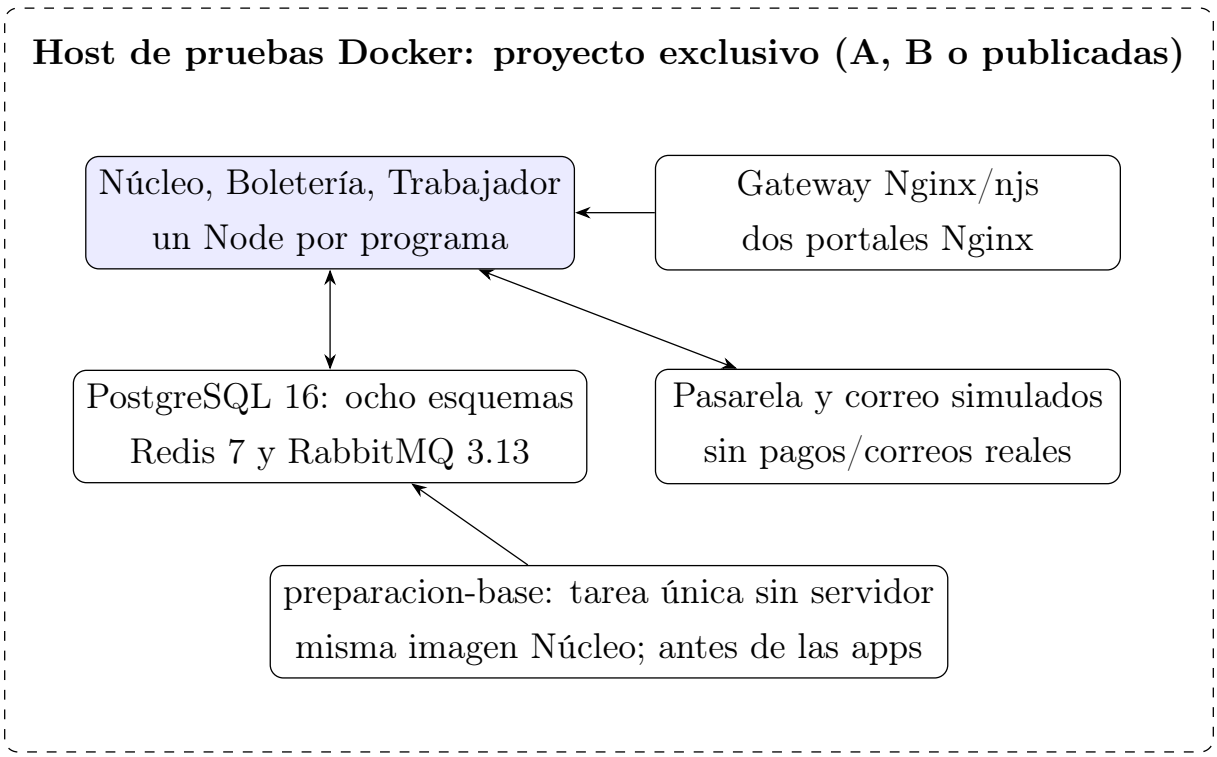



Figura 5: Despliegue comprobado en pruebas; no representa el entorno habitual migrado.

Docker Compose ejecuta los tres programas, gateway, PostgreSQL, Redis, RabbitMQ, dos proveedores simulados y dos portales. `preparacion-base` es una tarea sin listener: ejecuta las migraciones antes de habilitar ambas apps. No es un cuarto servidor. El override remoto separa datos y procesos; escalado replica Boletería; registro reutiliza la misma imagen Núcleo en preparación. Configuración válida no demuestra arranque en dos máquinas ni capacidad, redundancia o RNF-15. No hay Kubernetes, MongoDB, buscador ni réplica SQL desplegados por esta migración. Los diagramas de clúster anteriores son históricos, no evidencia de implementación.

## 7. Modelo de datos vigente

Un destino PostgreSQL común `POSTGRES_BD=hexacore`; las conexiones TypeORM siguen nombradas. Cada pool tiene solo su esquema, pg_catalog y extensiones en search_path. No se mezclan propietarios por semejanza de tablas.

| Conexión | Esquema | Historial / migraciones |
|---|---|---|
| identidad | identidad | migraciones / 3 |
| alimentos | alimentos | migraciones / 2 |
| personal-logistica | personal_logistica | migraciones / 2 |
| entradas-ventas | entradas_ventas | migraciones / 5 |
| entradas-catalogo | entradas_catalogo | migraciones / 1 |
| entradas-postventa | entradas_postventa | migrations / 1 |
| entradas-acceso-qr | entradas_acceso_qr | migraciones / 3 |
| finanzas-pagos | finanzas_pagos | migraciones / 1 |

Tabla 2: Ocho esquemas e historiales; 18 fuentes originales conservadas.

Ambos programas usan `finanzas_pagos.transacciones_pago`, sin claves por proceso ni duplicación del libro. Extensiones compartidas en `extensiones`; procedencia/SHA en `migracion_control`. synchronize/migrationsRun=false. Una preparación serializada reconoce la transferencia sin reaplicar DDL o instala las 18 migraciones en vacío. El mapa tabla por tabla está en la guía de consolidación. La copia transferida y la base vacía se prueban por separado; no se escribe sobre las bases habituales. Respaldos independientes son coherentes entre dominios solo tras inmovilizar conjuntamente escritores/callbacks/consumidores, no por ser pg_dump.

# 8. Reparto, evidencia y pendientes

Daniel: CU-001–005/021–023; Samuel: CU-006–010/024–026; Sebastián: CU-011–015/027–029; Diego: CU-016–020/030–032. Ubicaciones y pendientes en `GUIA-EQUIPO-MIGRACION.md`; instalación/pruebas en `App/backend/README.md`. F12/F13/F14/F15 no están resueltos. No se acredita rendimiento, disponibilidad mensual, alertas operativas ni nuevas garantías financieras. CI/CD y digests reales se registran con revisión/enlaces en ESTADO-MIGRACION, no se infieren de diagramas o YAML.